

Well formed formulas

A formula G is called well formed (WFF) if

$$G := At \mid (G_1 \odot \dots \odot G_n) \mid (\neg G_1) \mid \top \mid \perp \quad \odot \in \{\wedge, \vee, \leftrightarrow\}$$

where At is an atomic proposition (also called atom or bool. variable) and G_i are themselves WFFs

Subformulas

Given a formula G , every WFF Q in G is called a subformula

For a WFF $G = Q_2 \odot \dots \odot Q_n$

- $G|_x = G$
- $G|_i = Q_i$
- if $Q_i = Q'_1 \odot \dots \odot Q'_m$ then $G|_{i,j} = Q'_{i,j}$
- if $Q_i = \neg Q'$ then $G|_{i,1} = Q'$
- let π be a position i.j. ... where a subformula occur, then $A|_\pi = B$ denotes that subformula B occurs in A at position π

Splitting

Given a formula G :

$G_p^{+/1}$ denotes the formula where all occurrences of p in formula G are replaced by $\top/\perp$

- Lemma:** $I \models p \iff I(G) = I(G_p^{+/1})$
 $I \not\models p \iff I(G) = I(G_p^{+/1})$

- Theorem:** G is sat $\iff G_p^{+/1}$ is sat $\vee G_p^{+/1}$ is sat

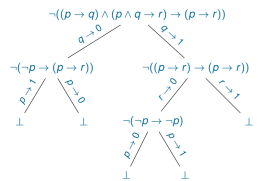
Splitting Algorithm

```

procedure split( $G$ )
parameters: function select
input: formula  $G$ 
output: "satisfiable" or "unsatisfiable"
begin
   $\bar{G} := \text{simply}(G)$ 
  if  $G = \top$  then return "satisfiable"
  if  $G = \perp$  then return "unsatisfiable"
   $(p, b) := \text{select}(G)$ 
  case  $b$  of
    1  $\rightarrow$ 
      if  $\text{split}(G_p^{+/1}) = \text{"satisfiable"}$ 
        then return "satisfiable"
      else return split( $G_p^{+/1}$ )
    0  $\rightarrow$ 
      if  $\text{split}(G_p^{+/1}) = \text{"satisfiable"}$ 
        then return "satisfiable"
      else return split( $G_p^{+/1}$ )
  end

```

Example:



Splitting Algorithm (with pure atoms)

```

procedure split( $G$ )
parameters: function select
input: formula  $G$ 
output: "satisfiable" or "unsatisfiable"
begin
   $\bar{G} := \text{simply, with pure atoms}(G)$ 
  if  $G = \top$  then return "satisfiable"
  if  $G = \perp$  then return "unsatisfiable"
   $(p, b) := \text{select}(G)$ 
  case  $b$  of
    1  $\rightarrow$ 
      if  $\text{split}(G_p^{+/1}) = \text{"satisfiable"}$ 
        then return "satisfiable"
      else return split( $G_p^{+/1}$ )
    0  $\rightarrow$ 
      if  $\text{split}(G_p^{+/1}) = \text{"satisfiable"}$ 
        then return "satisfiable"
      else return split( $G_p^{+/1}$ )
  end

```

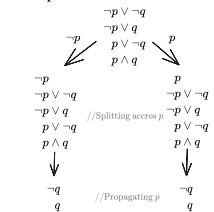
DPLL (splitting + propagation)

```

procedure DPLL( $S$ )
input: set of clauses  $S$ 
output: satisfiable or unsatisfiable
parameters: function select.literal
begin
   $S := \text{propagate}(S)$ 
  if  $S$  is empty then return satisfiable
  if  $S$  contains  $\square$  then return unsatisfiable
   $L := \text{select.literal}(S)$ 
  if DPLL( $S \cup \{L\}$ ) = satisfiable
    then return satisfiable
  else return DPLL( $S \cup \{\neg L\}$ )
end

```

Example:



Splitting and propagating q would result in formulas containing only the empty clause $\square$ which are always unsat

Entailment

Given a finite set of formulas $\Gamma = \{G_1, G_2, \dots\}$:

$$\Gamma : (I(\Gamma) = 1 \iff \forall G \in \Gamma : I(G) = 1)$$

$$\Gamma \models Q \iff \forall I : (\forall G \in \Gamma : I(G) = 1) \implies I(Q) = 1$$

$$\Gamma \models Q \iff \Gamma \wedge \neg \text{is unsat}$$

$$\text{Note that } \neg(\Gamma \models Q) = (\Gamma \wedge \neg Q)$$

Equivalent Replacement + Rewrite Rules

Given a formula G with subformula Q_1 denoted as $G[Q_1]$:
 $G[Q_2]$ would then denote the formula G where subformula Q_1 has been replaced with Q_2

- Lemma:** $Q_1 \leftrightarrow Q_2 \iff G[Q_1] \leftrightarrow G[Q_2]$

- Lemma:** $I \models p \iff I \models p \leftrightarrow \top$
 $I \not\models p \iff I \models p \leftrightarrow \perp$

Formula evaluation (using rewrite rule system)

```

procedure evaluate( $G, I$ )
input: formula  $G$ , interpretation  $I$ 
output: the value  $I(G)$ 
begin
  forall atoms  $p$  occurring in  $G$ 
    if  $I \models p$ 
      then replace all occurrences of  $p$  in  $G$  by  $\top$ ;
      else replace all occurrences of  $p$  in  $G$  by  $\perp$ ;
  rewrite  $G$  into a normal form using the rewrite rules,
  if  $G = \top$  then return 1 else return 0
end

```

Polarity

The polarity of a formula G and its subformulas is defined as:

- $\text{pol}(G, \varepsilon) = 1$ (this means that the input formula always needs to have polarity of 1)

- if $G|_\pi = \bigwedge_i Q_i$ then

$$\text{pol}(G, \pi, i) = \text{pol}(G, \pi)$$

- if $G|_\pi = \bigvee_i Q_i$ then

$$\text{pol}(G, \pi, i) = -\text{pol}(G, \pi)$$

- if $G|_\pi = Q_1 \rightarrow Q_2$ then

$$\text{pol}(G, \pi, 1) = -\text{pol}(G, \pi)$$

$$\text{pol}(G, \pi, 2) = \text{pol}(G, \pi)$$

- if $G|_\pi = Q_1 \leftrightarrow Q_2$ then

$$\text{pol}(G, \pi, 1) = \text{pol}(G, \pi, 2) = 0$$

- if $G|_\pi = \neg Q$ then

$$\text{pol}(G, \pi, 1) = -\text{pol}(G, \pi)$$

- if $G[Q]|_\pi$ and $\text{pol}(G, \pi) = 1$ then Q is a positive occurrence

- if $G[Q]|_\pi$ and $\text{pol}(G, \pi) = -1$ then Q is a negative occurrence

Coloring Algorithm

- Color in blue all arcs below in $\leftrightarrow$
- Color in red all uncolored arcs going down from $\rightarrow$ or left side of $\rightarrow$

Example:

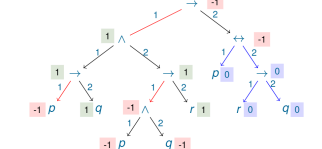


Tableau Inferences Rules

$f : A \vee B$	$t : A \vee B$	$f : A \wedge B$	$t : A \wedge B$
$f : A$	$t : A \mid t : B$	$f : A \mid f : B$	$t : A$
$f : B$			$t : B$

$f : A \rightarrow B$	$t : A \rightarrow B$	$f : \neg A$	$t : \neg A$
$t : A$	$f : A \mid t : B$	$t : A$	$f : A$
$f : B$			

Given a formula G the first entry of the tableau is of the form $t : G$. From there, all further entries are derived using the inference rules.

- Rules like $t : A \vee B$ produce two branches.

- A branch is closed if it contains a contradiction $f : A$ and $t : A$. The tableau is closed if all branches are closed.

- If no more inference rules can be applied to a branch and it didn't close, then it provides an interpretation satisfying the formula

Refutation (using tableau)

To (dis)prove a statement $A \models B$, one can use the tableau rules on the formula $\neg(A \rightarrow B)$ until:

- The tableau is closed, in which case the statement holds.

- No more inferences can be applied, in which case the statement does not hold.

Connectives Priority (to avoid parenthesis):

$$1. \neg, 2. \wedge, 3. \vee, 4. \rightarrow, 5. \leftrightarrow$$

Example:

$$\neg A \wedge B \rightarrow C \vee D \leftrightarrow E$$

is read as: $((\neg A) \wedge B) \rightarrow (C \vee D) \leftrightarrow E$

Monotonic Replacement

Given a formula G with subformula Q_1 at position π denoted as $G[Q_1]_\pi$:
 $G[Q_2]_\pi$ would then denote the formula G where subformula Q_1 at position π has been replaced with Q_2

- Lemma:** Given formulas G, Q_1, Q_2 where $I \models Q_1 \rightarrow Q_2$ and $G[Q_1]$ if $\text{pol}(A, \pi) = 1$ then $I \models G[Q_1]_\pi \rightarrow G[Q_2]_\pi$ if $\text{pol}(A, \pi) = -1$ then $I \models G[Q_2]_\pi \rightarrow G[Q_1]_\pi$

- Theorem:** Given G, Q_1, Q_2 such that $Q_1 \rightarrow Q_2$ is valid and $Q_1 = G|_\pi$ if $\text{pol}(G, \pi) = 1$ then $G[Q_1]_\pi \rightarrow G[Q_2]_\pi$ if $\text{pol}(G, \pi) = -1$ then $G[Q_2]_\pi \rightarrow G[Q_1]_\pi$

Pure Atoms

Given a formula G , an atom p is pure if all occurrences of p in A have the same polarity $\neq 0$

- Lemma:** Given $I \models G$ and G has pure atom p . If p has only positive occurrences then $I + \{p \mapsto 1\} \models G$ G is satisfiable $\iff G = G_p^+$

- has only negative occurrences then $I + \{p \mapsto 0\} \models G$ G is satisfiable $\iff G = G_p^-$

Pure Literals

Given a literal L occurring in a set of clauses S , L is called pure if S contains no clause $C_i = \left(\overline{L} \vee \bigwedge_{i,j} L_{i,j}\right)$.

Optimization:

If a pure literal L occurs in S then all clauses containing L can be satisfied by assigning the suitable truth value to L and can thus be removed.

Unit Propagation

Given a set of clauses $S = L \wedge \bigwedge_i C_i$.

Let S' be the set of clauses where all clauses C containing L are removed and all $C_i = \left(\overline{L} \vee \bigwedge_{i,j} L_{i,j}\right)$ are replaced

by $C'_i = \left(\bigvee_j L_{i,j}\right)$. Then it holds that $S \iff S' \cup \{L\}$

Inferences

Inferences δ are rules of form:

$$\delta = \frac{\varphi, \psi_1, \dots, \psi_n}{\chi}, \text{ where:}$$

- φ is the prerequisite of δ
- $\psi_1, \dots, \psi_n$ are justifications of δ
- χ is the consequence of δ

One can also write: $\{\varphi, \psi_1, \dots, \psi_n\} \xrightarrow[\text{KB}]{\chi} \chi$

Modus Ponens:

$$\frac{A \rightarrow B \quad A}{B} \text{MP}$$

Read as: If A then B , A holds, therefore B must hold

Resolution Inference:

Applied to clausal form formulas $S = \bigwedge_i C_i$ where two

clauses $C_i = L \vee \bigwedge_{i,j} L_{i,j}, C_i' = \overline{L} \vee \bigwedge_{i',j'} L_{i',j'}$ are

resolved into one clause $C_{i \vee i'} = \bigvee_j L_{i,j} \vee \bigvee_{j'} L_{i',j'}$

Example

$$\frac{C \vee p \quad \neg p \vee D}{C \vee D}$$

$\neg C \vee D$ is the resolvent, p is the resolved atom

$$\frac{C \vee p \vee I \quad \neg C \vee I}{C \vee I}$$

Derivations

Let $\text{KB} = \{C_1, \dots, C_n\}$ be a set of input (given) clauses

A derivation (from KB) is a sequence of the form

$$C_1, \dots, C_n, C_{n+1}, \dots, C_m, \dots$$

- such that for every $n \geq k + 1$
 - C_n is a resolvent of C_i and C_j , for some $1 \leq i, j < n$, or
 - C_n is a factor of C_i , for some $1 \leq i < n$.

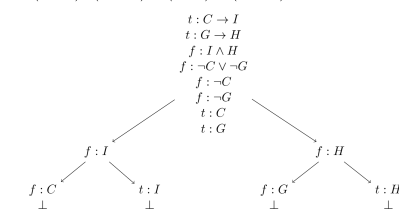
The empty disjunction, or empty clause, is written as $\square$

A refutation (of KB) is a derivation from KB that contains $\square$

Example

Given the statement $\frac{(C \rightarrow I) \wedge (C \rightarrow H) \wedge \neg(I \wedge H)}{A} \models \frac{\neg C \vee G}{B}$

One can (dis)prove this statement by using the tableau on $\neg(A \rightarrow B)$ i.e. $(C \rightarrow I) \wedge (C \rightarrow H) \wedge \neg(I \wedge H) \wedge \neg(\neg C \vee G)$



The tableau is closed and thus the entailment is proven

Clauses and Literals

A literal is an atom p or its negation $\neg p$. Literal $\bar{p} = \neg p$

A clause is a disjunction of literals $\bigvee_{i=1}^n l_i$

An empty clause $\square$ ($n = 0$) is always negative

A clause with one literal ($n = 1$) is a unit clause

A clause with at most one positive literal is a horn clause

CNF Transformations

$$A \leftrightarrow B \implies (\neg A \vee B) \wedge (\neg B \vee A)$$

$$A \rightarrow B \implies \neg A \vee B$$

$$\neg(A \wedge B) \implies \neg A \vee \neg B$$

$$\neg(A \vee B) \implies \neg A \wedge \neg B$$

$$\neg\neg A \implies A$$

$$\left(\bigwedge_{i=1}^n A_i\right) \vee \bigvee_{j=1}^n B_j \implies \bigwedge_{i=1}^n \left(A_i \vee \bigvee_{j=1}^n B_j\right)$$

Definition Translation + Clausal Form

Naming Lemma

Given a set of formulas S and a formula B . Let n be variable not occurring in S nor in $B \in S$. Let S' be the set of formulas where B has been replaced by n . Then S is sat $\iff S' \cup \{n \wedge B\}$ is sat.

A clausal form of a formula A is a set of clauses A' where:

$$A \iff A'$$

A clausal form of a set of formulas S is a set of clauses S' where:

$$S \iff S'$$

Example of Definitional Clause Form Translation

(unoptimized version)			
	subformula	definition	clauses
n_1	$\neg((p \rightarrow q) \wedge (p \wedge q \rightarrow r) \rightarrow (p \rightarrow r))$	$n_1 \leftrightarrow \neg n_2$	n_1 $\neg n_1 \vee \neg n_2$ $n_1 \vee n_2$
n_2	$(p \rightarrow q) \wedge (p \wedge q \rightarrow r) \rightarrow (p \rightarrow r)$	$n_2 \leftrightarrow (n_3 \vee n_7)$	$\neg n_2 \vee \neg n_3 \vee n_7$ $n_2 \vee n_3$ $\neg n_2 \vee n_7$
n_3	$(p \rightarrow q) \wedge (p \wedge q \rightarrow r)$	$n_3 \leftrightarrow (n_4 \wedge n_5)$	$\neg n_3 \vee n_4$ $\neg n_3 \vee n_5$ $\neg n_4 \vee \neg n_5 \vee n_3$
n_4	$p \rightarrow q$	$n_4 \leftrightarrow (p \rightarrow q)$	$\neg n_4 \vee \neg p \vee q$ $p \vee n_4$ $\neg q \vee n_4$
n_5	$p \wedge q \rightarrow r$	$n_5 \leftrightarrow (n_6 \rightarrow r)$	$\neg n_5 \vee n_6 \vee r$ $n_5 \vee n_6$ $\neg r \vee n_5$
n_6	$p \wedge q$	$n_6 \leftrightarrow (p \wedge q)$	$\neg n_6 \vee \neg p \vee \neg q$ $\neg n_6 \vee q$ $\neg p \vee \neg q \vee n_6$
	$p \rightarrow r$	$n_7 \leftrightarrow (p \rightarrow r)$	$\neg n_7 \vee \neg p \vee r$ $p \vee n_7$ $\neg r \vee n_7$